

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Generics & Collections

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

Generics

- Generics allow types to be used as parameters of a class.
- They differ from traditional method parameters, which specify the type of a variable.
- It is possible to declare both generic methods and generic classes.A generic class is a class that depends on an unspecified type (in the example, *E*).

Generics

- I generics consentono di avere tipi come parametri di una classe.
- Sono differenti dai parametri tradizionali dei metodi che specificano il tipo di una varibile
- E' possibile dichiarare metodi generici e classi generiche.
- Una classe generica e' una classe che dipende da un tipo non specificato (nell' esempio E).

```
1 public class GenericTest<E>{
2 private E var;
3 public GenericTest(E var){
4 this.var=var;
5 }
6 public E getVar() {
7 return var;
8 }
9 public void setVar(E var) {
10 this.var=var;
11 }
12 }
```

Generics

- When defining a variable of a generic class, it is necessary to specify the actual type that will replace the declared type parameter.
- Any reference type can be used (primitive types are not allowed).

```
1 public class GenericTest<E>{
2 private E var;
3 public GenericTest(E var){
4 this.var=var;
5 }
6 public E getVar() {
7 return var;
8 }
9 public void setVar(E var) {
10 this.var=var;
11 }
12 }
```

```
GenericTest<Integer> intero = new GenericTest<Integer>(10);
15 GenericTest<String> stringa = new GenericTest<String>("Ciao mondo!");
```

Generics

- Similarly, it is possible to define a generic method.
- Generic methods are methods that use generic types for their parameters.
- The idea is similar to that of generic classes; however, the scope of the type is limited to the individual method.

```
1 class PrintArrayGeneric
2 {
3 public <E > void printArray ( E el []){
4 //...
5 }
6 }
```

Generics

The wildcard character (`?`) can be used to represent an unspecified type.

The wildcard can be used as:

- the type of a parameter,
- the type of a field,
- the type of a local variable,
- the return type.

Generics

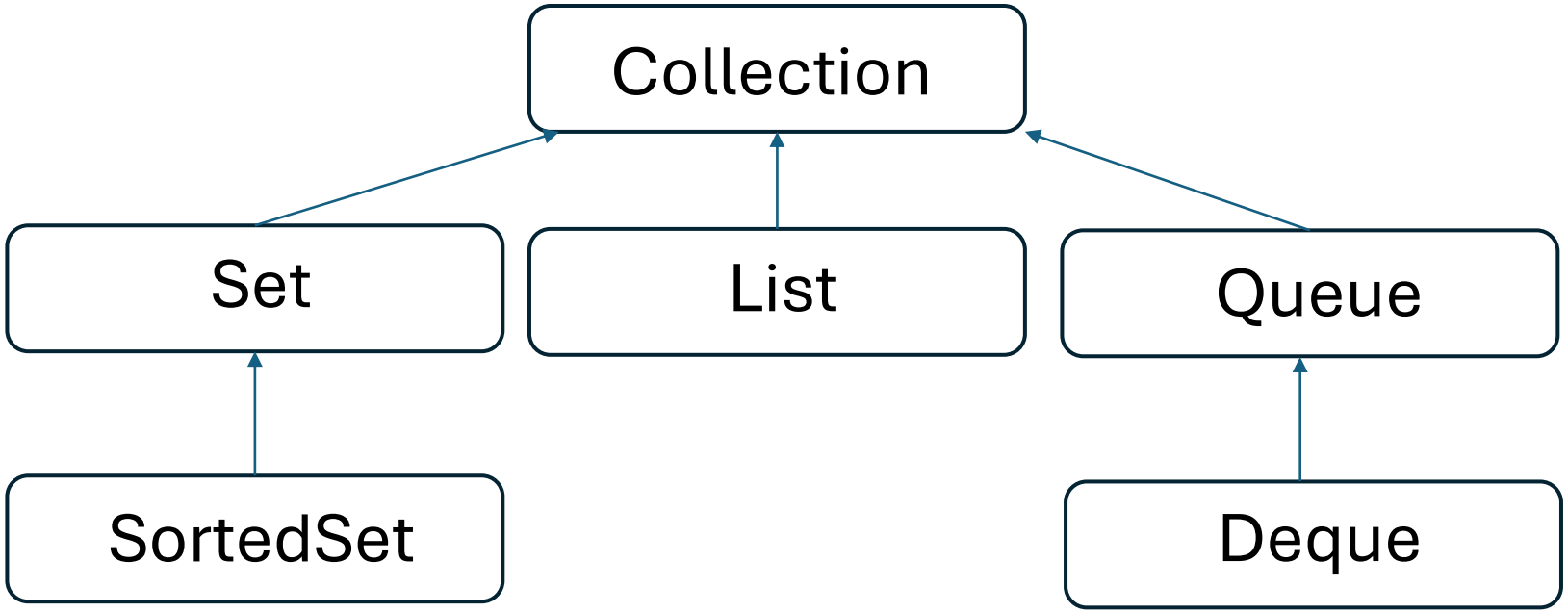
- It is possible to constrain the type specified by a wildcard to be a subtype of a given type.
- The method shown can handle any collection whose elements extend the class **SuperClass**

```
public void method(Collection<? extends SuperClass> collection);
```

- It is also possible to constrain the type specified by a wildcard to be a supertype of a given type.
- The method shown can handle any collection whose elements are supertypes (parents) of the class **SubClass**.

```
public void method(Collection<? super SubClass> collection);
```

Java Collections Framework



- All such classes are dedicated structures to store objects

Basic Operations

```
1 int size();
2 boolean isEmpty();
3 boolean contains(Object element);
4 boolean add(E element);// Optional
5 boolean remove(Object element);// Optional
6 Iterator iterator();
7
8 boolean containsAll(Collection<?> c); //Collection<?> shortcut per
Collection<? extends Object>
9 boolean addAll(Collection<? extends E> c);// Optional
10 boolean removeAll(Collection<?> c);// Optional
11 boolean retainAll(Collection<?> c);// Optional
12 void clear();// Opzionale
```

Set

- A **Set** represents a collection of objects with no duplicate elements.
- In general, it does not preserve any ordering, except in the case of the **SortedSet** interface.
- Some common implementations include:
  - HashSet
  - TreeSet
  - LinkedHashSet

List

- A **List** represents a sequence of objects that can contain duplicate elements.
- Some common implementations include::
  - ArrayList
  - LinkedList
  - Vector

Operazioni Basilari

```
1 E get(int index);
2 E set(int index, E element);
3 boolean add(E element);
4 void add(int index, E element);
5 E remove(int index);
6 boolean addAll(int index, Collection<?extendsE> c);
7
8 int indexOf(Object o);
9 int lastIndexOf(Object o);
10
11 ListIterator<E> listIterator();
12 ListIterator<E> listIterator(int index);
```

Queue

- A **Queue** represents a queue of objects that can contain duplicate elements.
- In general, it does not preserve any ordering, except in the case of the **SortedQueue** interface.
- One implementation is:
  - LinkedList
- The **Deque** subtype represents a queue with both ends accessible (Double-Ended Queue).

Operazione	Lancia Eccezione	Valore Speciale
Insert	add(e)	offer(e)
Remove	remove()	poll()
Examine	element()	peek()

Map

- A **Map** represents an association of values with keys.
- It does not allow duplicate keys but permits null values.
- Some common implementations include:HashMap
  - TreeMap
  - LinkedHashMap

Operazioni Basilari

```
1 V put(K key, V value);
2 V get(Object key);
3 V remove(Object key);
4 boolean containsKey(Object key);
5 boolean containsValue(Object value);
6 int size();
7 boolean isEmpty();
89
Set<K> keySet();
10 Collection<V> values();
11 Set<Map.Entry<K,V>> entrySet();
```

HashTable

- A **Hashtable** is an implementation of a **Map**.
- It does not allow null values.
- It is synchronized.

```
1 Hashtable<String, Integer> numbers
2 = new Hashtable<String, Integer>();
3 numbers.put("one", 1);
4 numbers.put("two", 2);
5 numbers.put("three", 3);
```

Iterator <E> Interface

- The **Iterator<E>** interface allows traversing and removing objects from a collection.
- It consists of the following methods:

```
1 boolean hasNext()
2 E next();
3 void remove();
```

▶ Example

```
1 Iterator<String> iterator = list.iterator();
2 while(iteratore.hasNext()){
3 string = iterator.next();//Next Element
4 iterator.remove();//Remove current element
5 }
```

Iterable <T> Interface

- The **Iterable<T>** interface allows the implementation of the enhanced for-loop.
- It consists of the following method:

Iterator<T> iterator();

The enhanced for-loop can be used on objects that implement this interface.

Exercise: Stack<E>

Implement the **Stack<E>** class.
The class should:

1. Throw a checked exception when attempting to push onto a full stack.
2. Throw a checked exception when attempting to pop from an empty stack.
3. Allow the use of the enhanced for-loop on the stack itself. **Note:** this implies iterating over the stack, not over one of its attributes.
4. Provide implementations for the following methods:

pushAll:

1. **Input:** a stack of elements that are subtypes of **E**
2. **Behavior:** pushes the elements onto the stack
3. **Output:** void

pushAllIn:

1. **Input:** a stack of elements that are supertypes of **E**
2. **Behavior:** pushes the elements into the input stack
3. **Output:** void